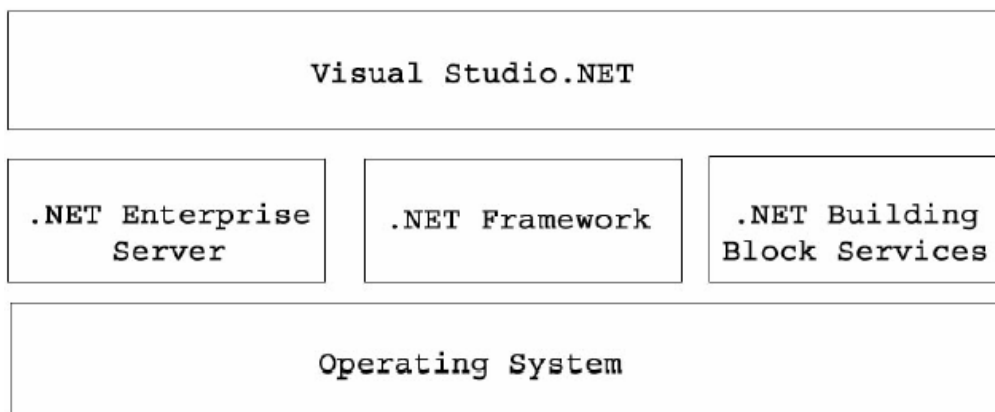


บทที่ 3

Microsoft .NET Technology

3.1. .NET Platform

องค์ประกอบของแพลตฟอร์ม .NET แบ่งออกได้ 3 ชั้น



รูปที่ 3-1 องค์ประกอบ .NET Platform

1.1 ชั้นระบบปฏิบัติการ (Operating System)

1.2 ชั้นถัดมาแบ่งออกเป็น 3 ส่วน ได้แก่

- .NET Enterprise Server
 - SQL Server 2000, BizTalk Server, Exchange 2000
- .NET Framework
 - เป็น Framework ที่สำคัญในการพัฒนา .NET
- .NET Building Block Services
 - Microsoft passport, Notification and Messaging, Personalization , XML, Store, Calendar, Directory and Search

1.3 ชั้น Visual Studio .NET เครื่องมือในการพัฒนา .NET

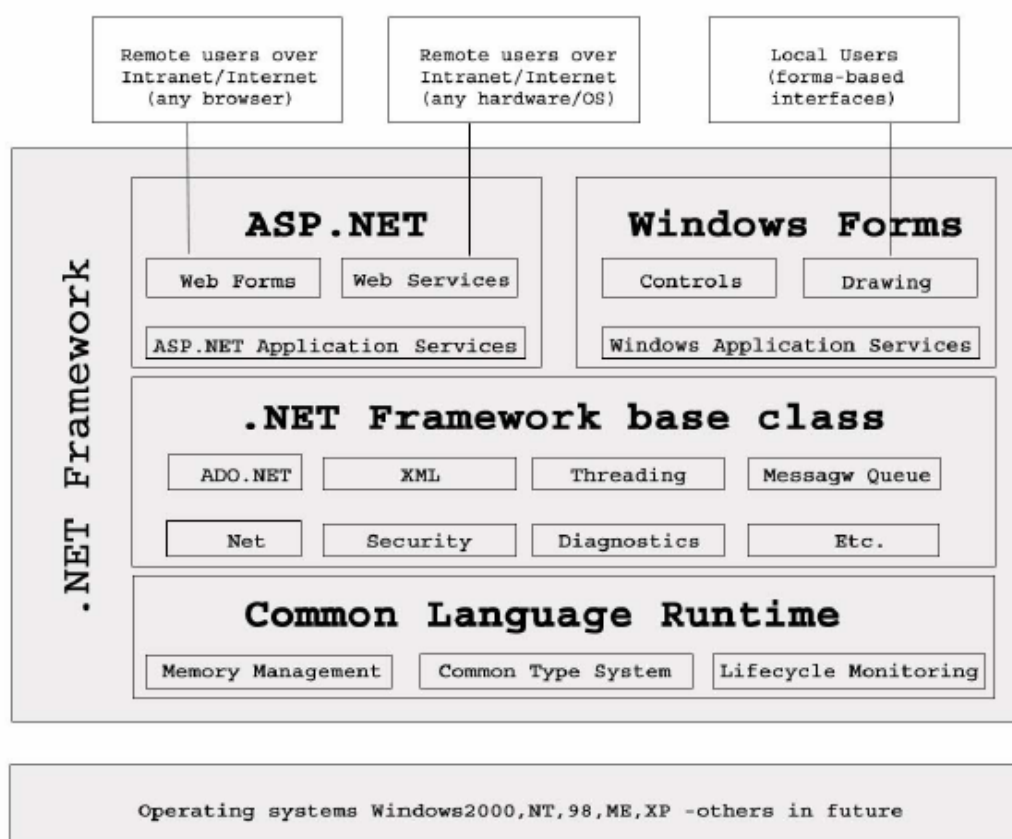
3. 2. .NET Framework

.NET Framework มีองค์ประกอบ ดังนี้

1. Common Language Runtime (CLR) มีหน้าที่ในการจัดเตรียมบริการต่าง ๆ ที่เกี่ยวข้อง เพื่อให้โปรแกรมที่ขอใช้บริการสามารถทำงานได้

2..NET Framework Class Library (Unified programming classes) คือ ไฟล์ไลบรารีที่ทำหน้าที่จัดเก็บและรวบรวมข้อมูลที่เป็นต้องเรียกใช้งาน ในการพัฒนาโปรแกรม

3. ASP.NET เป็น Web application model ซึ่งประกอบไปด้วยกลุ่มของ control และ infrastructure ที่ใช้ในการสร้าง ASP Web applications , ASP.NET ประกอบไปด้วยส่วนของ control ซึ่งรวมถึงส่วนของ HTML ด้วย ส่วนของ control รันบนเซิร์ฟเวอร์ ส่วน HTML รันบน browser บนเซิร์ฟเวอร์นั้นส่วน control จะถูกเขียนในรูปแบบของ object-oriented programming , ASP.NET ยังให้บริการส่วน infrastructure ด้วยเช่น session state management และ process recycling ยิ่งไปกว่านั้น ASP.NET ก็ใช้หลักการนี้ในการพัฒนาจากซอร์ฟแวร์เป็นเซอร์วิส โดยผ่านกลไกสำคัญคือ XML เราสามารถเขียน business logic และใช้ ASP.NET infrastructure เพื่อให้บริการผ่าน SOAP



รูปที่ 3-2 องค์ประกอบ .NET Framework

3.2.1 Common Language Runtime (CLR)

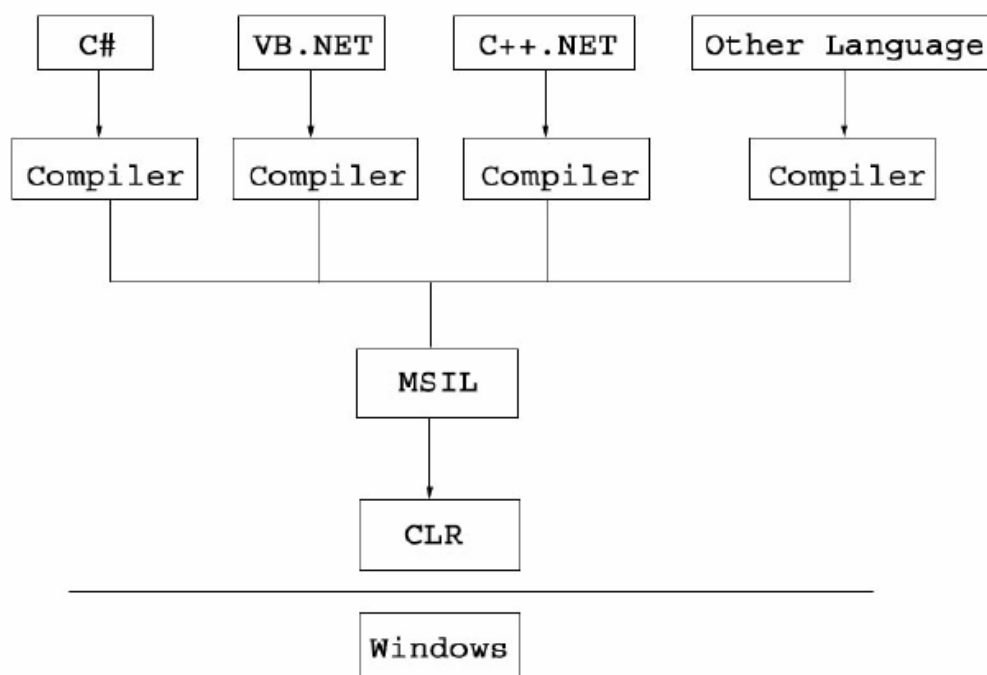
CLR จะทำหน้าที่เป็นตัวกลางในการจัดเตรียมบริการและทรัพยากรสำหรับรองรับการประมวลผล และการทำงานของโปรแกรมประยุกต์ที่ทำงานบนเทคโนโลยีของ .NET เช่น การจัดการหน่วยความจำ, ความปลอดภัยในการเข้ารหัสโปรแกรม ด้วยความสามารถของ CLR ทำให้เราสามารถพัฒนาโปรแกรมโดยไม่ต้องขึ้นกับระบบปฏิบัติการ

การทำงานของโปรแกรมนั้น เริ่มจากคอมไพเลอร์ของแต่ละภาษาจะคอมไพล์โค้ดให้เป็นแบบ Microsoft's Intermediate Language (MSIL) หรือเรียกสั้นๆว่า IL Code ซึ่งจะมีลักษณะคล้ายภาษา Assembly จาก IL Code ก็แปลงเป็นโปรแกรมที่รันโดย CLR อีกทีหนึ่ง

นักพัฒนาสามารถที่จะเลือกได้ว่าเราจะ build แอปพลิเคชันไปอยู่ในรูปของ .exe หรือ MSIL ซึ่งจะกลายเป็น Just-In-Time(JIT) คือเมื่อเราได้แอปพลิเคชันในรูปแบบของ MSIL แล้ว เมื่อรันโปรแกรมใช้งานจริง มันจะถูกคอมไพเลอร์ JIT ทำการคอมไพล์โค้ด MSIL ในส่วนที่ต้องการใช้ไปเป็น Native Code อีกทีซึ่งนำไปให้เครื่องรันต่อ หากมีการใช้โค้ดในส่วนเดิมอีกก็ไม่ต้องมีการคอมไพล์ซ้ำ

การแปลงโค้ด MSIL ไปเป็น Native Code ประโยชน์ที่เราจะได้คือ

1. เราสามารถแปลงไปเป็น Native Code ที่เหมาะสมกับระบบปฏิบัติการนั้นรันอยู่
2. เราสามารถใช้ Native Code ที่ใช้ความสามารถของ CPU ได้อย่างเต็มที่ เช่น ใน CPU P4 เราสามารถใช้คำสั่งในส่วนของ SSE2 ได้



รูปที่ 3-3 รูปแบบการคอมไพล์โค้ดไปเป็น IL Code

3.2.1.1 Memory Management

สำหรับ .NET แล้วจะมี Garbage Collector (GC) ซึ่งทำหน้าที่คอยรวบรวมหาหน่วยความจำที่ถูกทิ้งไว้เป็นขยะ แต่ไม่มีใครอ้างอิงเรียกใช้แล้ว โดยการทำงานจะแตกต่างจากใน VB Runtime รุ่นเก่าที่ให้ Object รับผิดชอบในการจัดการ Garbage Collector เป็นผู้ดูแลจัดการทั้งหมด โดย GC จะเรียก method ชื่อ Object.Finalize โดย

อัตโนมัติเมื่อ object เลิกใช้งานการจัดการหน่วยความจำที่ดีกว่าของ .NET เกิดขึ้นเมื่อ object ถูกสร้างขึ้นมา CLR จะจัดการกันพื้นที่ส่วนหนึ่งไว้สำหรับโปรแกรมที่เรียกว่า Heap โดยเริ่มต้นจากพื้นที่หน่วยความจำที่ว่าง จากนั้นก็อ้างอิงชี้ตำแหน่งไปยังส่วนบนสุดของหน่วยความจำ เมื่อหน่วยความจำถูกใช้แล้วก็เลยก็เลยไปเรื่อยๆ จะเหลือพื้นที่ว่างเป็นช่วงๆ ซึ่ง GC จะเข้ามาทำการ Compact หน่วยความจำเสียใหม่ ลองนึกถึงหลักการทำงานแบบเดียวกันกับตอนที่ทำ Defragment บนฮาร์ดดิสก์

3.2.1.2 Common Type System

ใน .NET จะมี common Type System ที่จะให้ทุกภาษามี Types ที่เหมือนกัน เป็นมาตรฐาน โดยที่ทุก Types ที่สนับสนุนโดย Common Types System นั้น จะสืบทอดคุณสมบัติมาจาก System.object ดังนั้นคุณจะพบว่า Object ทั้งหลายส่วนมากแล้วจะสนับสนุน Method เหล่านี้คือ

Equal(Object) = Boolean

GetHashCode() = Int32

GetType () = Type

Tostring () =String

ข้อดีของ Common Language Runtime

- ไม่มีปัญหาเรื่องการรัน บน MS Windows Platform ต่างๆ เนื่องจาก CLR จะทำการตรวจสอบระบบปฏิบัติการให้โดยอัตโนมัติ และทำการจำลองสภาพให้เหมาะสมกับการทำงานของโปรแกรมโดยอัตโนมัติ เช่นเดียวกัน

- ไม่ต้องสนใจเรื่อง Registry การเข้าถึงคอมโพเนนต์ต่างๆที่เกี่ยวข้องจะอยู่ในความดูแล ของ CLR ทั้งหมด เราจำเป็นต้องรู้แค่จะใช้ Namespace ตัวใดในการอ้างอิงถึงเท่านั้น

- รองรับการพัฒนาจากหลายภาษา ในปัจจุบันเราสามารถใช้อย่างน้อยภาษาสามภาษาทำงานร่วมกันได้ด้วยเทคโนโลยีของ COM แต่อย่างไรก็ตาม เนื่องจากโปรแกรมยังผูกติดกับภาษาอยู่ ความเร็วของโปรแกรมที่ได้ก็จะแตกต่างกัน ขึ้นอยู่กับความสามารถของ Compiler ภาษานั้นๆ Microsoft Common Language Specification จะทำให้ Compiler ทุกตัวที่ใช้ .NET ทำการแปลภาษาไปยังเป้าหมายเดียวกัน คือ การทำงานกับ CLR ได้อย่างไรปัญหาส่งผลให้ โปรแกรมที่ได้จากและภาษามีผลลัพธ์ที่เท่าเทียมกันเมื่อทำงานอยู่ภายใต้การดูแลของ CLR

- นำ Source code มาใช้งานใหม่ได้ COM ยังถูกจำกัดการ reuse ในวงแคบเนื่องจากการผูกติดกับระบบปฏิบัติการ .NET แยกออกได้ 3 รูปแบบได้แก่

1. เราสามารถเขียน คลาส ด้วยภาษาหนึ่ง สามารถเรียกใช้ผ่านอีกภาษาหนึ่งได้
2. คลาสสามารถ Inherit ข้ามภาษาได้
3. เว็บเซอร์วิส เป็นอีกรูปแบบหนึ่งโดยที่เสนอบริการให้แก่ทุกภาษาที่อ้างอิง CLR ได้

- ชนิดของข้อมูลมีประสิทธิภาพ จากข้อผิดพลาดเกี่ยวกับตัวแปรเช่น ประกาศตัวแปรขนาด 10 ไบต์ แต่ทำงานจริงกลับมีขนาด 20 ไบต์ ถ้าไม่มีการเขียนโปรแกรมดักจับ โปรแกรมจะทำงานผิดพลาด ใน .NET สิ่งเหล่านี้

จะไม่เกิดขึ้น CLR จำลองการทำงานทำงานของ code อย่างใกล้ชิดก่อนที่จะประมวลผลขั้นสุดท้าย CLR จะไม่อนุญาตให้โปรแกรมใช้ตัวแปรที่มีการผิดพลาดเหล่านั้น

-การ Debug ที่มีประสิทธิภาพ CLR จะทำการแยกส่วนของโปรแกรมเป็นส่วนๆ ทำให้การตรวจสอบข้อผิดพลาดทำได้ง่ายสะดวกและรวดเร็ว การแก้ไขสามารถแก้ไขเฉพาะส่วนที่ผิดพลาดได้โดยไม่รบกวนส่วนอื่นๆ ของโปรแกรม และเนื่องจาก CLR จัดการกับทุกภาษาด้วยกัน ข้อผิดพลาดต่างๆ ที่แสดงออกมาในแต่ละภาษาจะอยู่ในรูปแบบเดียวกัน

-ระบบจัดสรรทรัพยากรที่ดีขึ้น การดำเนินการที่เกี่ยวข้องกับทรัพยากรต่างๆ ระหว่างโปรแกรม ไม่ว่าจะเป็นระบบการจัดการไฟล์ระบบการเชื่อมต่อ หรือพื้นที่ว่างของหน่วย เป็นต้น จะถูกดูแลจัดการอย่างใกล้ชิด โดย CLR ไม่สามารถมีโปรแกรมตัวใดตัวหนึ่งครอบครองทรัพยากรไว้แต่เพียงผู้เดียว

-การแจกจ่ายโปรแกรมง่ายขึ้น การติดตั้งโปรแกรมแบบเดิม โปรแกรมหนึ่งๆมีไฟล์เป็นจำนวนมาก ,ต้องมีการลงทะเบียนใน registry ,การสร้าง short cut , การจัดเตรียมระบบที่เกี่ยวข้องใน .NET การติดตั้งโปรแกรมเป็นไปโดยง่าย เนื่องจาก CLR จะทำการแยกโปรแกรมออกเป็น ส่วนๆ การติดตั้งโปรแกรมจึงเป็นการคัดลอก ไฟล์ที่เกี่ยวข้องมาไว้บน hard disk เท่านั้น ในทางกลับกัน การยกเลิกการติดตั้ง ก็เพียงลบไฟล์เหล่านั้นออก

-ระบบความปลอดภัยที่ดีขึ้น ระบบรักษาความปลอดภัยใหม่นี้จะทำการตรวจสอบพฤติกรรมการทำงานของโปรแกรมอย่างใกล้ชิดภายในระบบเดิมนั้น การทำงานของภาษาสคริปต์ต่างๆหรือโปรแกรมที่มาจากอินเทอร์เน็ต อาจเป็นโปรแกรมที่สร้างช่องโหว่ให้กับระบบได้

3.2.2 .NET Framework Class Library

ไฟล์ไลบรารีที่ทำหน้าที่จัดเก็บและรวบรวมข้อมูลที่เป็นต้องเรียกใช้งาน ในการพัฒนาโปรแกรม ซึ่งไฟล์เหล่านี้ เป็นไลบรารีร่วมกันในทุกภาษา ส่งผลให้เราสามารถ inheritance ,error handling, debugging ข้ามภาษาได้ และการที่ .NET Framework เสนอวิธีการสำเร็จรูปผ่าน Class Library เหล่านี้ทำให้ รูปแบบการเขียนโปรแกรมเปลี่ยนไป โดยมีความเป็นระบบ และเป็นลักษณะของ OOP เพิ่มขึ้น เช่นใน VB.NET ต้องมีการอ้างอิงถึง class library ที่เกี่ยวข้องและต้องการใช้งาน

3.2.3 Presentation Layer

การแสดงผลสามารถทำได้ 2 รูปแบบหลัก คือในรูปแบบ Windows form และ Web form ซึ่งก่อนหน้านี้แยกทั้งคำสั่ง และกระบวนการเขียนโปรแกรม ของทั้ง 2 แบบ แต่ใน .Net แล้วได้รวมทั้งสองอย่างมาทำให้เหมือนกันที่สุดคือเราเขียนโปรแกรมบน Windows อย่างไร คุณก็เขียน Web Application ด้วยวิธีเดียวกัน โดยเรียกใช้คลาส ด้วยคำสั่งที่เหมือนกัน บน Windows จะอยู่ใน คลาส ที่ชื่อ system.windows แต่ Web จะอยู่ในคลาส ชื่อ system.web.ui ซึ่งวิธีใช้เหมือนกันมาก ที่พิเศษคือ มี Object สำหรับ Web ให้คุณเขียนโปรแกรม แบบ visual basic ธรรมดา แต่เมื่อรันบน browser แล้วจะสร้าง tag html และ javascript ให้อัตโนมัติ ทำให้ browser ทั่วๆไปก็สามารถรันได้ ไม่จำเป็นต้องสนับสนุน .NET โดยเฉพาะ ตัวอย่างเช่น เราสร้างเว็บเพจที่มีปฏิทินด้วย ในส่วนของรูปแบบ web จะแสดงในรูปของ html code ส่วนปฏิทินจะอยู่ในรูปของ java script ซึ่งทางโปรแกรมจะทำการแปลงให้ออก

3.3 .NET Compact Framework

.NET Compact Framework คือ platform ของ Microsoft .NET ที่ใช้ในการพัฒนาอุปกรณ์เล็กๆ ความจริงแล้ว .NET Compact Framework ก็คือ .NET Framework ที่ตัดเอาส่วนที่ไม่สำคัญหรือไม่ได้ใช้ออกไป เพื่อให้ขนาดเล็กลงจนขนาดที่สามารถ install ลงบนอุปกรณ์เล็กๆ ที่ไม่ใช่คอมพิวเตอร์ได้ เช่น โทรศัพท์มือถือ เป็นต้น คุณสมบัติหลักของ .NET Compact Framework

- .NET Compact Framework ถูกออกแบบให้เป็น platform ในการพัฒนา สำหรับการเขียนและเรียกใช้ XML เว็บเซอร์วิส
- เนื่องจาก .NET Compact Framework ใช้ programming model เดียวกันกับ desktop .NET Framework จึงง่ายต่อ developer ในการเขียน application ใหม่บน .NET Compact Framework
- ใช้ Visual Studio .NET ตัวเดียวกับที่ใช้ใน desktop .NET Framework ในการเขียน program device application
- ใช้ระบบ evidence-based security คืออนุญาตให้เฉพาะ application ที่ผ่านการรับรองจากผู้ผลิต device นั้นจึงจะสามารถ access secure resource ได้ ถ้าเป็น application ที่ยังไม่ผ่านการรับรองความปลอดภัย จะ access ได้เพียง basic system resource เท่านั้น
- ออกแบบสำหรับอุปกรณ์ที่มี resource จำกัด เช่น โทรศัพท์มือถือ โดยที่ตัว framework เองใช้ memory อย่างมีประสิทธิภาพ และมีระบบคืน resource ที่ไม่ได้ใช้แล้วกลับมาจาก application อย่างมีประสิทธิภาพ
- ใช้ Just-in-time compiler ซึ่งมีประสิทธิภาพดีกว่า ระบบ device programming อื่นๆ ที่ใช้ code interpreter
- เพิ่มประสิทธิภาพและลดต้นทุนในการพัฒนา เนื่องจากใช้ programming model และ toolset ของ Visual Studio .NET อันเดียวกันกับของ desktop .NET framework ทำให้นักพัฒนาสามารถใช้ความรู้ความชำนาญที่มีอยู่แล้วมาพัฒนา application บนอุปกรณ์ได้

3.4 .NET Framework on Linux

ไมโครซอฟท์ ทำการส่งสเปกตรัมมาตรฐานของ .NET Framework ให้แก่ ECMA เพื่อสร้างเป็นมาตรฐานสากล ส่งผลให้ .NET Framework เป็น Language Independent และ Platform Independent ตอนนี้มีบริษัท XIMIAN ทำการสร้าง framework บน Linux แล้ว ประกอบไปด้วย CLR , BASE CLASS , C# Compiler

3.4.1 .NET Framework Runtime (CLR)

Ximian ตั้งชื่อ project ในการทำ .NET Framework บน Linux ว่า Mono โดยจะมีหลักอยู่ 3 โปรแกรม ดังนี้

1. mint โปรแกรมนี้คือโปรแกรม .NET Framework Runtime บน Linux โดยสามารถเอาโปรแกรมที่เขียนโดยใช้ .NET Framework บน Windows มาใช้งานได้เลย

2. pedump เป็นโปรแกรมที่ดูโครงสร้างของแฟ้ม .exe ที่เป็น Managed code
3. monodis เป็นโปรแกรมที่แสดงให้เห็นถึงภาษา MSIL ของ .exe นั้น

3.4.2 NET Framework based class Library

การ Implement Library ของ .NET นั้น มีความซับซ้อนยิ่งกว่าส่วนอื่นๆ หากสนใจสถานะว่า Library ตัวใดทำไปแล้วมากน้อยเพียงใด สามารถหาข้อมูลได้จาก <http://www.go-mono.com/classstatus/index.html>

3.4.3 C# Compiler

Ximian ทำคอมไพเลอร์ภาษา C# ที่ชื่อว่า mcs (Mono CSharp compiler)

3.5 มาตรฐาน CLI และ C# ของ ECMA

เดือนสิงหาคม ปี 2000 Microsoft , HP และ Intel ได้ร่วมเป็นผู้สนับสนุนร่วมเสนอการกำหนด รายละเอียด สำหรับ CLI และ ภาษา C# ให้กับองค์กรการจัดมาตรฐานระหว่างประเทศ ECMA ผลคือ ECMA ได้ตั้งมาตรฐาน กลุ่มการทำงานขึ้นมา 2 กลุ่ม สำหรับ CLI และ C# คือ TC39-TG3 และ TC39-TG2 ตามลำดับซึ่งเป็นคณะกรรมการ เทคนิคซึ่งรับผิดชอบเกี่ยวกับภาษาในการเขียนโปรแกรม และการพัฒนาแอปพลิเคชัน

ระหว่างปีถัดมาผู้ให้การสนับสนุนร่วมกับสมาชิก ECMA อื่นๆ และ ผู้รับเชิญ (รวมถึง IBM Fujitsu Software, Plum Hall, Monash University และ ISE) ได้ร่วมทำการกลั่นกรองรายละเอียด เป็นมาตรฐานใน ธันวาคม ปี 2001 สมัชชาสหประชาชาติ ECMA อนุมัติ มาตรฐานฉบับที่ 1 ของ C# และ CLI เป็น ECMA-334 และ ECMA-335 ตามลำดับ รายงานเกี่ยวกับ CLI ได้ถูกอนุมัติเป็นมาตรฐาน ECMA TR84 ด้วย